



Universidad Nacional Experimental De Guayana

Vicerrectorado Académico.

Coordinación General De Pregrado.

Proyecto De Carrera: Ingeniería Informática

Lenguajes y Compiladores

Grupo: Control de Flujo

Autómatas Finitos Determinísticos (AFD) y Autómatas Finitos No Determinísticos (AFN).

Profesor:

Marquez Felix

Integrantes:

Canchis Victor 27935276

Gonzalez Blas 31074243

Muñoz Diosdado 30932108

Rojas Celimar 31981398

Ciudad Guayana, mayo de 2026

Tabla de Contenido

Tabla de Contenido.....	2
Tabla de Figuras	4
Introducción	5
Autómatas finitos determinísticos (AFD)	6
Funcionamiento de un AFD	7
Características de los AFD	8
<i>Determinismo.....</i>	<i>8</i>
<i>Número finito de estados</i>	<i>8</i>
<i>Procesamiento secuencial.....</i>	<i>8</i>
<i>Existencia de un único estado inicial.....</i>	<i>8</i>
<i>Estados de aceptación definidos.....</i>	<i>9</i>
<i>Ausencia de ambigüedad</i>	<i>9</i>
Representación de los AFD	9
Ejemplo Técnico de Aplicación.....	10
<i>Definición de la Quintupla:</i>	<i>10</i>
<i>Tabla de Transiciones:</i>	<i>10</i>
Autómatas Finitos No Deterministas (AFND).....	12
Definición Formal y Componentes	12
El Núcleo del No Determinismo: La Función de Transición (δ)	13

Análisis Comparativo: Ambigüedad y Funciones Matemáticas.....	14
Representación y Equivalencia Teórica.....	15
Análisis Comparativo: AFD vs. AFND	16
Diferencias Formales en la Función de Transición (δ).....	16
Procesamiento y Manejo de la Ambigüedad	17
El Rol de las Transiciones Vacías (ϵ).....	17
Equivalencia y el Teorema de Rabin-Scott.....	18
Cuadro Comparativo.....	19
Aplicaciones	20
Aplicación de los autómatas finitos deterministas (AFD)	20
<i>Reconocimiento de lenguaje</i>	<i>20</i>
<i>Diseño de compiladores.....</i>	<i>20</i>
<i>Procesamiento de texto</i>	<i>21</i>
<i>Protocolos de red.....</i>	<i>21</i>
Aplicación de los autómatas finitos deterministas (AFND)	21
<i>Motores de Búsqueda y Expresiones Regulares</i>	<i>21</i>
<i>Validación de Formatos y Datos</i>	<i>22</i>
<i>Filtros de Spam y Detección de Intrusos</i>	<i>22</i>
Conclusión	23
Referencias.....	24

Tabla de Figuras

<i>Figure 1. Diagrama de transición</i>	<i>11</i>
<i>Figure 2. Múltiples transiciones definidas</i>	<i>13</i>
<i>Figure 3. Diagrama AFD</i>	<i>14</i>
<i>Figure 4. Diagrama AFND.....</i>	<i>15</i>
<i>Figure 5. Demostración teorema de Rabin-Scott</i>	<i>18</i>

Introducción

La teoría de la computación y el estudio de los lenguajes formales constituyen pilares fundamentales en las ciencias de la computación, proporcionando las bases teóricas necesarias para el procesamiento de información y el diseño de software complejo. En el centro de esta disciplina se encuentran los autómatas finitos, modelos matemáticos diseñados para procesar cadenas de símbolos e identificar patrones lógicos a través de un número limitado de estados.

El presente trabajo de investigación se centra en el análisis detallado de las dos variantes principales de estos modelos: los Autómatas Finitos Determinísticos (AFD) y los Autómatas Finitos No Determinísticos (AFND). A lo largo de este documento, se desglosará su estructura matemática formal basada en una quintupla definitoria $M = (Q, \Sigma, \delta, q_0, F)$ y se examinará cómo la naturaleza de su función de transición (δ) marca la diferencia fundamental entre el comportamiento estricto y secuencial del AFD y la capacidad de bifurcación y paralelismo conceptual del AFND.

Asimismo, se explorará la relación directa entre ambos a través del Teorema de Rabin-Scott, el cual demuestra su equivalencia computacional y justifica su uso complementario en la ingeniería moderna. Finalmente, se abordarán sus aplicaciones prácticas en el mundo real, demostrando cómo estos conceptos teóricos son la base tecnológica de motores de búsqueda, analizadores léxicos en compiladores, expresiones regulares y sistemas de ciberseguridad.

Autómatas finitos determinísticos (AFD)

Es uno de los modelos matemáticos más importantes para el estudio de los lenguajes formales y los procesos computacionales; está compuesto por un número finito de estados, cuya función es procesar cadenas de símbolos pertenecientes a un alfabeto determinado. Como señalan John Hopcroft et al. (2007), "un autómata finito determinístico se define por el hecho de que, para cada estado y para cada símbolo de entrada, hay exactamente una transición a un estado siguiente". Esta propiedad garantiza un comportamiento preciso y predecible, ya que el autómata siempre seguirá un único camino posible durante el procesamiento de la cadena, evitando cualquier tipo de ambigüedad.

Un AFD se define formalmente como un modelo matemático compuesto por cinco elementos esenciales

$$M = (Q, \Sigma, \delta, q_0, F)$$

- **Q (Conjunto de estados):** Es un inventario finito de todas las condiciones o situaciones posibles del sistema. Aunque el número de estados es finito, la complejidad del comportamiento del autómata depende de cómo estos se interconectan.
- **Σ (Alfabeto):** Es el conjunto finito de símbolos (bits, caracteres, señales) que el autómata está diseñado para interpretar.
- **δ (Función de transición):** Es el componente crítico, definido como $\delta: Q \times \Sigma \rightarrow Q$.

En un AFD, esta función es total y unívoca, lo que significa que para cada par estado -

símbolo existe un resultado único. No se permiten transiciones vacías o espontáneas (ϵ).

- **q0 (Estado inicial):** El punto de entrada único del proceso. Representa la configuración del sistema antes de procesar cualquier dato.
- **F (Conjunto de estados finales o de aceptación):** Es un subconjunto de Q que marca el éxito de la operación. Si al agotar la cadena de entrada el autómata se encuentra en uno de estos estados, se dice que la cadena pertenece al lenguaje que el autómata reconoce.

Funcionamiento de un AFD

El funcionamiento de un Autómatas Finitos Determinísticos se basa en la lectura secuencial de una cadena de entrada de izquierda a derecha. El autómata inicia en el estado inicial q_0 y, por cada símbolo leído, aplica la función de transición δ para cambiar de estado.

El proceso continúa hasta consumir todos los símbolos de la cadena. Al finalizar, el autómata verifica el estado en el que terminó:

- Si pertenece al conjunto de estados de aceptación F, la cadena es aceptada.
- Si no pertenece a dicho conjunto, la cadena es rechazada.

Decimos entonces que una cadena w es aceptada por el autómata cuando, después de consumir el último símbolo de la cadena, el estado resultante pertenece al conjunto de aceptación. Matemáticamente, este proceso se describe mediante la función de transición extendida δ^* , encargada de evaluar el recorrido completo de la cadena dentro del autómata.

Características de los AFD

Los Autómatas Finitos Determinísticos poseen diversas características que los convierten en uno de los modelos más importantes dentro de la teoría de autómatas y los lenguajes formales.

Determinismo

La principal característica de un AFD es su comportamiento determinístico. Esto significa que, para cada estado y cada símbolo del alfabeto, existe únicamente una transición posible. Debido a esta propiedad, el autómata nunca presenta dudas ni múltiples caminos durante el procesamiento de una cadena. Cada entrada produce siempre el mismo recorrido de estados.

Número finito de estados

Los AFD poseen una cantidad limitada de estados. Esto permite representar sistemas de reconocimiento de patrones de forma organizada y controlada.

Procesamiento secuencial

El autómata procesa las cadenas símbolo por símbolo, siguiendo el orden en que aparecen dentro de la entrada. A medida que se leen los símbolos, el AFD realiza transiciones entre estados hasta consumir completamente la cadena.

Existencia de un único estado inicial

Todo AFD debe poseer un único estado inicial desde el cual comienza el procesamiento de cualquier cadena. Esto permite que todas las ejecuciones del autómata

tengan un punto de partida claramente definido, asegurando consistencia en el reconocimiento del lenguaje.

Estados de aceptación definidos

Los AFD cuentan con uno o varios estados de aceptación encargados de determinar si una cadena pertenece al lenguaje reconocido. La presencia de estos estados permite establecer condiciones claras para aceptar o rechazar cadenas procesadas.

Ausencia de ambigüedad

En un AFD no existen múltiples caminos posibles para procesar una misma entrada. Cada símbolo conduce a un único estado determinado. Gracias a esto, el comportamiento del autómata es completamente preciso y fácil de analizar.

Representación de los AFD

La arquitectura de un AFD se puede visualizar mediante dos métodos complementarios que facilitan su análisis y diseño. Estos métodos permiten verificar que el autómata cumple con la propiedad de determinismo (un solo camino posible).

- **Diagramas de Transición:** Grafos dirigidos donde los nodos son estados y las aristas etiquetadas representan las transiciones. Los estados finales se representan mediante un doble círculo, como se observa en la Figura 1.
- **Tablas de Transición:** Mientras que el diagrama es visual, la tabla es una matriz donde las filas representan los estados actuales y las columnas los símbolos del

alfabeto, permitiendo una visualización rápida de todos los destinos posibles de la función δ .

Ejemplo Técnico de Aplicación

Para ilustrar los conceptos anteriores, se presenta un AFD diseñado para reconocer el lenguaje

$$L = \{\text{cadenas sobre } \{0, 1\} \text{ que terminan en } 01\}.$$

Definición de la Quintupla:

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- $q_0 = q_0$ (Estado inicial)
- $F = \{q_2\}$ (Acepta cuando detecta la secuencia "01")

Tabla de Transiciones:

Estado Actual	Entrada: 0	Entrada: 1
q_0 (Inicio)	q_1	q_0
q_1	q_1	q_2 (Aceptación)
q_2 (Final)	q_1	q_0

En el ejemplo presentado, el sistema inicia en q_0 y solo avanza hacia q_1 al detectar un 0, asegurando que se cumpla el requisito mínimo del lenguaje. Una vez que la cadena recibe un 1 estando en q_1 , el autómata alcanza el estado de aceptación q_2 (identificado con doble círculo). Si la lectura finaliza exactamente en este punto, la cadena es validada; de lo contrario, cualquier símbolo adicional obliga al sistema a retroceder de estado según la tabla de transiciones para reevaluar la secuencia, garantizando un control riguroso y determinista en todo momento.

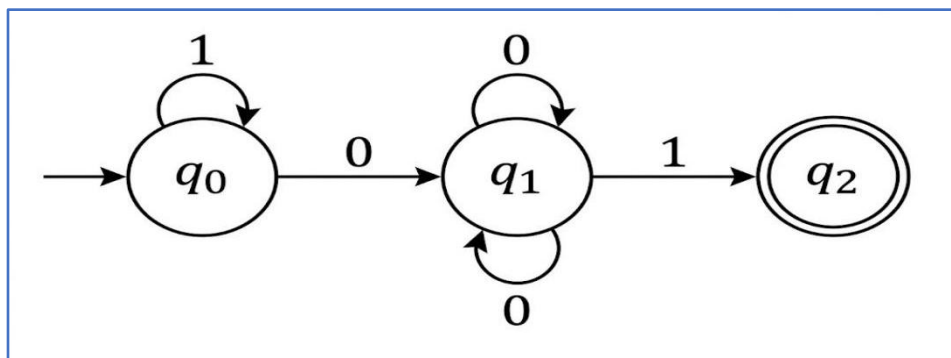


Figure 1. Diagrama de transición

Autómatas Finitos No Deterministas (AFND)

En el ámbito de la teoría de la computación y los lenguajes formales, los autómatas finitos representan modelos matemáticos esenciales para el procesamiento de información, el análisis léxico y el reconocimiento de patrones en cadenas de caracteres. Dentro de esta categorización teórica, el Autómata Finito No Determinista (AFND) destaca por introducir el concepto de bifurcación y multiplicidad de caminos computacionales. Un Autómata Finito No Determinista es una quintupla compuesta por 5 elementos.

Definición Formal y Componentes

La estructura matemática que rige a un Autómata Finito No Determinista se expresa mediante la siguiente quintupla formal:

$$A = (Q, \Sigma, \delta, q_0, F)$$

Para comprender la naturaleza del modelo, es necesario desglosar cada uno de los elementos que conforman esta expresión:

- **Q:** Corresponde al conjunto de estados del autómata, representando las posibles situaciones que se pueden dar cuando este procese una cadena.
- **Σ :** Es el alfabeto que contiene los símbolos con los que trabaja el autómata, es decir, los inputs que este mismo acepta. Por ejemplo, un alfabeto binario clásico se definiría como

$$\Sigma: \{0,1\}$$
- **q_0 :** Es el estado inicial representado por una flecha, y desde este estado comienza a procesarse cualquier cadena.

- **F:** Representa el conjunto de estados finales. Son aquellos que, si al terminar de procesar una cadena nos encontramos en ellos, el autómata devolverá la aceptación de esta cadena como resultado. En la representación mediante grafos, este tipo de estados se representa con dos círculos en lugar de uno.

Es fundamental señalar que, hasta este punto de la definición formal, todo permanece exactamente igual que en los Autómatas Finitos Deterministas (AFD).

El Núcleo del No Determinismo: La Función de Transición (δ)

La divergencia teórica y funcional entre un modelo determinista y uno no determinista recae exclusivamente en la función δ . Esta función recoge el conjunto de transiciones definidas desde cada uno de los estados del autómata, constituyendo la principal diferencia entre un AFD y un AFND.

La restricción fundamental de los modelos deterministas dicta que, en los AFD, desde un estado solo puede haber una transición definida para cada símbolo como máximo. En contraposición, en los AFND, desde un estado puede haber varias transiciones definidas para cada símbolo.

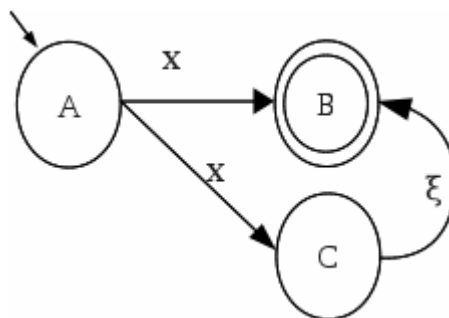


Figure 2. Múltiples transiciones definidas

Análisis Comparativo: Ambigüedad y Funciones Matemáticas

Para ilustrar esta diferencia, podemos observar el comportamiento del procesamiento de datos. En un AFD completo, en cada estado recibamos un 0 o un 1 solo podemos tomar un camino, o sea no hay ninguna ambigüedad en el procesamiento. Sin embargo, si añadimos una transición extra en cualquier estado (como el inicial), pasaría a ser un Autómata Finito No Determinista.

Al realizar esta modificación geométrica y conceptual, ya no se cumple la restricción que desde un estado solo puede haber una transición definida para cada símbolo del autómata. Con esta transición extra, cuando el autómata reciba un 0 como input no está claro qué camino escoger, y por eso es llamado no determinista.

Si analizamos las definiciones matemáticas rigurosas de la función δ para cada uno de los autómatas, podemos apreciar de dónde nace esta cualidad:

- **En el caso del AFD:** La función δ se define como una función de Q y Σ hacia el mismo Q ($\delta: Q \times \Sigma \rightarrow Q$). Es decir, que para un estado y un símbolo, llegamos siempre a otro estado único.

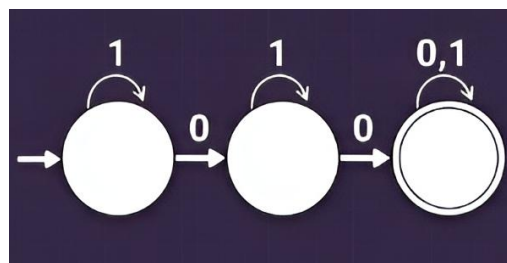


Figure 3. Diagrama AFD

- **En el caso del AFND:** La función δ está definida como una función hacia 2^Q ($\delta: Q \times \Sigma \rightarrow 2^Q$). En notación matemática, esto representa el conjunto de todos los subconjuntos posibles de Q , es decir, el conjunto de partes de Q . Esto implica que el destino de una transición no es un solo estado de forma obligatoria; este subconjunto puede ser un estado, dos o todos los estados simultáneamente.

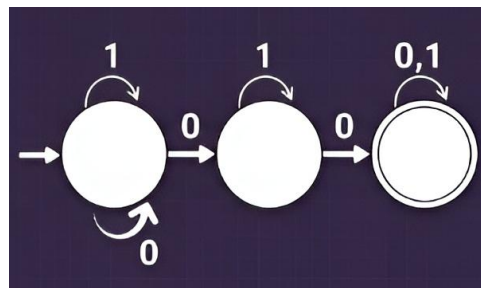


Figure 4. Diagrama AFND

Representación y Equivalencia Teórica

A pesar de la complejidad conceptual que añade el conjunto de partes 2^Q , a nivel visual y de diseño, la representación de un AFND es igual a la de un AFD y se puede hacer por tablas de transiciones o grafos.

Finalmente, a nivel de equivalencia matemática, el modelo no determinista engloba al determinista. Dado que la función δ de un AFND permite que un subconjunto destino contenga un único estado (emulando exactamente el comportamiento estricto de $(\delta: Q \times \Sigma \rightarrow Q)$), se establece el axioma fundamental de que todo AFD es, por definición, un AFND.

Análisis Comparativo: AFD vs. AFND

Aunque los Autómatas Finitos Determinísticos (AFD) y los No Determinísticos (AFND) reconocen exactamente la misma clase de lenguajes (los lenguajes regulares), difieren de forma sustancial en la definición de su función de transición, en su mecánica de procesamiento y en la complejidad de su diseño. A continuación, se detallan las divergencias clave que definen la utilidad de cada modelo.

Diferencias Formales en la Función de Transición (δ)

Como se definió previamente, la naturaleza de la función de transición es lo que separa a ambos modelos.

- **En el AFD:** La función δ es total y unívoca. Para cada par estado-símbolo existe exactamente un estado sucesor bien definido: $\delta: Q \times \Sigma \rightarrow Q$. Esta propiedad garantiza que, ante cualquier cadena de entrada, el autómata sigue una ruta de cómputo única, asegurando una ejecución en tiempo lineal $O(|w|)$ respecto a la longitud de la cadena.
- **En el AFND:** Se amplía la expresividad mediante el uso del conjunto potencia 2^Q y la incorporación de la transición vacía ϵ . Formalmente se expresa como:
 $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$. Un par (q, a) puede dar lugar a cero, uno o múltiples estados sucesores, habilitando ramificaciones en el árbol de cómputo.

Procesamiento y Manejo de la Ambigüedad

- **Procesamiento Determinístico:** Dado una cadena $w = a_1 a_2 \dots a_n$, el AFD aplica iterativamente la función δ siguiendo una única trayectoria. Este mecanismo es libre de ambigüedad por construcción y resulta directamente traducible a una tabla de transiciones implementable en software.
- **Procesamiento No Determinístico (Paralelismo Conceptual):** El AFND explora todas las trayectorias posibles de forma simultánea. Conceptualmente, es como si el autómatata se clonara en cada bifurcación para explorar en paralelo todas las opciones. Esta ambigüedad lo hace ideal para el diseño, pero menos idóneo para la ejecución directa en software, ya que simular el paralelismo requiere mantener un conjunto de estados activos, incrementando el coste computacional.

El Rol de las Transiciones Vacías (ϵ)

A diferencia de los AFD, que no permiten transiciones espontáneas, los AFND pueden cambiar de estado sin consumir ningún símbolo de la entrada. Su presencia otorga ventajas significativas:

- **Modularidad:** Facilitan la unión y concatenación de subautómatas independientes mediante saltos desde el estado inicial hacia los subgrafos, sin necesidad de redefinir las transiciones existentes.
- **Reducción de complejidad:** Permiten representar lógicas complejas con un número significativamente menor de estados y arcos al evitar la materialización explícita de todas las rutas posibles.

Equivalencia y el Teorema de Rabin-Scott

A pesar de sus diferencias estructurales, ambos modelos poseen el mismo poder computacional. El **Teorema de Rabin-Scott (1959)** establece que para todo AFND existe un AFD equivalente, lográndose esto mediante la *Construcción de Subconjuntos*. En este procedimiento, cada estado del AFD equivalente corresponde a un subconjunto de estados del AFND original.

Esta conversión tiene un coste exponencial en el peor de los casos (un AFND de n estados puede generar un AFD de hasta 2^n estados). Sin embargo, justifica un flujo de trabajo estándar en la creación de compiladores:

- **AFND para el diseño:** Su compacidad es la herramienta natural para la especificación inicial de patrones léxicos y expresiones regulares.
- **AFD para la ejecución:** Una vez convertido a su forma determinística, garantiza un reconocimiento eficiente ideal para analizadores léxicos.

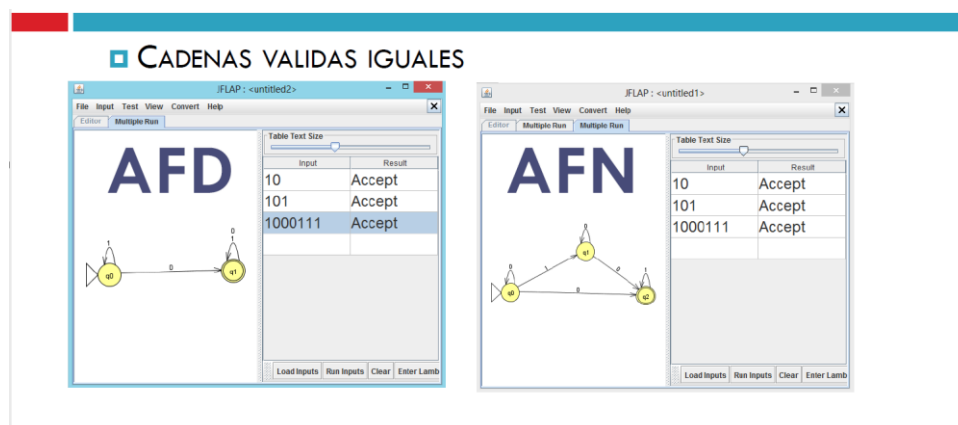


Figure 5. Demostración teorema de Rabin-Scott

Cuadro Comparativo

La siguiente tabla sintetiza los aspectos más relevantes de ambos modelos para su aplicación práctica en la ingeniería:

Aspecto	AFD (Determinístico)	AFND (No Determinístico)
<i>Determinismo</i>	<i>Total: $\delta(q, a)$ produce exactamente un estado.</i>	<i>No total: $\delta(q, a) \subseteq 2^Q$; acepta ϵ-transiciones.</i>
<i>Diseño humano</i>	<i>Complejo: requiere explicitar todas las transiciones.</i>	<i>Intuitivo y compacto; ideal para especificación.</i>
<i>Eficiencia de ejecución</i>	<i>Óptimo al garantizar un tiempo lineal $O(w)$</i>	<i>Mayor coste computacional</i>
<i>Tamaño del diagrama</i>	<i>Grande: hasta 2^n estados tras conversión.</i>	<i>Pequeño: ramificaciones implícitas reducen estados.</i>
<i>Aplicación típica</i>	<i>Analizadores léxicos, motores Regex, implementación.</i>	<i>Especificación de patrones, construcción de Thompson.</i>

Aplicaciones

Aplicación de los autómatas finitos deterministas (AFD)

Los autómatas finitos deterministas son un modelo matemático ampliamente utilizado en informática y otros campos para el reconocimiento de patrones en cadenas de caracteres. La principal ventaja de los AFD radica en su facilidad de comprensión e implementación, ya que la función de transición está completamente definida y el comportamiento del autómata está totalmente determinado por su estado actual.

Debido a esta propiedad, los DFA se ha utilizado ampliamente en diversas aplicaciones, tales como:

Reconocimiento de lenguaje

Una de las aplicaciones más comunes de los autómatas finitos deterministas (AFD) es el reconocimiento de lenguajes formales. Los AFD pueden diseñarse para reconocer un lenguaje específico, como un lenguaje de programación o un lenguaje de marcado, especificando el conjunto de cadenas aceptables en dicho lenguaje.

Diseño de compiladores

Los autómatas finitos deterministas (AFD) se utilizan en el diseño de compiladores, que son programas que traducen el código fuente escrito en un lenguaje de programación a código máquina. Los compiladores utilizan los AFD para analizar el código fuente e identificar los distintos tokens, como palabras clave e identificadores, que componen el lenguaje.

Procesamiento de texto

Los autómatas finitos deterministas (AFD) se utilizan en aplicaciones de procesamiento de texto, como correctores ortográficos y editores de texto, para reconocer patrones en el texto y realizar acciones específicas basadas en esos patrones.

Protocolos de red

Los DFA se utilizan en el diseño de protocolos de red, como TCP/IP, para especificar la secuencia de eventos que deben ocurrir durante la comunicación entre dos dispositivos. Ciberseguridad: Los DFA se utilizan en ciberseguridad para detectar y prevenir actividades maliciosas mediante el reconocimiento de patrones en el tráfico de red.

Aplicación de los autómatas finitos deterministas (AFND)

A diferencia de un sistema determinista, donde cada paso dicta un único camino a seguir, un AFND tiene la capacidad de explorar múltiples caminos simultáneamente o "adivinar" la transición correcta ante una entrada. Los AFND se utilizan ampliamente en:

Motores de Búsqueda y Expresiones Regulares

Cada vez que buscas una palabra en un documento grande, filtras correos electrónicos o usas una barra de búsqueda que admite comodines (como * o ?), estás utilizando el poder de los AFND. Las expresiones regulares (Regex) se compilan directamente en un AFND. Esto permite que el motor de búsqueda evalúe múltiples coincidencias y variaciones de una palabra al mismo tiempo, sin tener que retroceder y empezar de nuevo si un patrón falla a la mitad.

Validación de Formatos y Datos

Cuando interactúas con un formulario web o un sistema que requiere datos precisos, hay una máquina de estados operando detrás. Por ejemplo: Para verificar que un correo electrónico tenga el símbolo @, o asegurar que una contraseña cumpla con los requisitos de seguridad.

Filtros de Spam y Detección de Intrusos

Los sistemas de seguridad de redes y los proveedores de correo electrónico monitorean flujos masivos de datos en tiempo real. Utilizan AFND para buscar patrones de firmas de virus o características comunes de spam dentro del texto. El enfoque no determinista es ideal aquí porque permite al sistema rastrear miles de posibles patrones maliciosos simultáneamente a medida que los datos fluyen a través de la red, detectando amenazas antes de que lleguen a la bandeja de entrada o comprometan un sistema.

Conclusión

El análisis exhaustivo de los Autómatas Finitos Determinísticos (AFD) y No Determinísticos (AFND) demuestra que, si bien presentan diferencias radicales en su mecánica de procesamiento y diseño estructural, ambos modelos son herramientas matemáticamente equivalentes y complementarias. La dualidad entre el determinismo y el no determinismo no reside en el poder expresivo sino en su utilidad práctica durante las distintas fases del desarrollo de software.

Por un lado, el AFND destaca por su modularidad y su capacidad para manejar la ambigüedad, permitiendo explorar múltiples caminos simultáneamente de forma intuitiva. Esto lo convierte en el modelo ideal para la especificación inicial de patrones y el soporte de herramientas flexibles como las expresiones regulares. Por otro lado, el AFD brilla en la fase de ejecución; al estar libre de ambigüedad y procesar la información en un tiempo lineal óptimo, garantiza la eficiencia requerida en sistemas de alto rendimiento como los compiladores y los protocolos de red.

El Teorema de Rabin-Scott actúa como el puente definitivo entre ambas arquitecturas, permitiendo a los ingenieros diseñar con la libertad y compacidad del modelo no determinista, para luego transformarlo mediante la construcción de subconjuntos en un autómata determinista eficiente y listo para ser implementado en código. En definitiva, dominar la teoría y la aplicación de los autómatas finitos es una competencia técnica indispensable, ya que estos modelos matemáticos continúan siendo el motor silencioso que impulsa el procesamiento de texto, la validación de datos y la seguridad en la arquitectura del software contemporáneo.

Referencias

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Teoría de autómatas, lenguajes y computación 3/E*. ADDISON WESLEY.

Sipser, M. (2012). *Introduction to the Theory of Computation*. Cengage Learning.

Málaga, E. J. (2008). *Teorías de autómatas y lenguajes formales*.

GeeksforGeeks. (2023, 28 enero). Application of Deterministic Finite Automata (DFA).

Recuperado de <https://www.geeksforgeeks.org/theory-of-computation/application-of-deterministic-finite-automata-dfa/>

Viramontes, J. E. C. (2005). *Teoría de la computación*.

Client challenge. (s. f.). Recuperado de <https://www.slideshare.net/slideshow/automatas-finitos-70286440>

Rabin, M. O., & Scott, D. (1959). Finite Automata and Their Decision Problems. *IBM Journal Of Research And Development*, 3(2), 114-125.

<https://doi.org/10.1147/rd.32.0114>